



MASTER CPP OPPS

OBJECTIVE

The objective of these C++ OOP notes is to provide a comprehensive understanding of the key principles and features of ObjectOriented Programming, while covering a wide range of essential topics. Starting with an introduction to object-oriented design, its history, and advantages, learners will explore the core terminologies and features of OOP, including classes, objects, encapsulation, and access modifiers. The notes will delve into the lifecycle of objects through constructors and destructors, and differentiate between const and non-const functions. Advanced topics such as static data members, function and operator overloading, and class relationships (composition, aggregation, inheritance) will be covered, along with multiple inheritance and polymorphism. The exploration of abstract classes and interfaces will build a solid understanding of dynamic behavior in OOP. Furthermore, the notes will introduce the concepts of generic programming using function and class templates and the Standard Template Library (STL). Additional sections will cover object serialization through object streams, as well as exception handling to ensure robust and error-resilient code. This outline ensures that learners develop a deep and practical understanding of C++ OOP and its applications.

Malahim Haseeb

Roll no 516 , 0328 4671797

Abdullah Bashir

Roll no 558 , 0313 2914373

Page Numbers

Introduction-----	pg_2
Object-Oriented Programming (OOP)-----	pg_2
Access Specifiers-----	pg_3
Encapsulation-----	pg_5
Nesting of Member Functions-----	pg_7
Memory Allocation-----	pg_8
Arrays of objects-----	pg_9
Static Methods and Members-----	pg_10
Friend Function-----	pg_12
Friend function of another class-----	pg_14
Friend Class-----	pg_15
Constructors-----	pg_17
Constructors Overloading-----	pg_18
Constructors with default arguments-----	pg_19
Dynamic initialization of objects using Constructors-----	pg_19
Copy Constructor-----	pg_21
Destructor-----	pg_23
Inheritance and it types-----	pg_26
Ambiguity Resolution-----	pg_31
Const vs Non const Functions-----	pg_33
Initialization list in constructors-----	pg_35
Pointers-----	pg_38
Pointers to Objects-----	pg_40
this pointer-----	pg_41
Arrow operator ->-----	pg_43
Composition-----	pg_45
Aggregation-----	pg_47
Polymorphism-----	pg_49
Abstraction-----	pg_52
Interfaces-----	pg_54
Operator overloading-----	pg_56
Templates-----	pg_59
Template for multiple parameters-----	pg_61
Template with default parameters-----	pg_62
Member Function template & overloading template-----	pg_63
Standard Template Library (STL)-----	pg_65
Exceptional handling in cpp-----	pg_68

Introduction

Introduction to Object-Oriented Design:-

Object-Oriented Design (OOD) is a programming paradigm that structures software systems around objects rather than actions or logic. Each object represents a part of the application and contains both data (attributes) and methods (functions). OOD helps model real-world entities more intuitively and promotes reusability and modularity.

History of Object-Oriented Design:-

- 1960s: OOD originated in the 1960s with the development of the **Simula** language, designed for simulation, which introduced the concept of classes and objects.
- 1970s: In the 1970s, **Smalltalk** further refined object-oriented principles by including key features like inheritance and dynamic typing.
- 1980s: The paradigm gained more traction with the development of **C++** (an extension of C), integrating OOD into mainstream programming. Since then, languages like **Java**, **Python**, and **C#** have popularized OOD as the dominant design approach.

Advantages of Object-Oriented Design:-

- **Modularity:** Software components are divided into independent modules, improving manageability and clarity.
- **Reusability:** Classes and objects can be reused across programs or different parts of the same program.
- **Extensibility:** New features can be easily added by extending existing classes.
- **Maintainability:** Changes in one part of a system have minimal impact on others, reducing debugging effort.
- **Abstraction:** Complex systems can be modeled using simple, understandable concepts.

Object-Oriented Introduction Programming (OOP)

Class:-

A blueprint for creating objects. It defines the structure and behavior of objects. C++ was originally developed by Bjarne Stroustrup and was initially named "C with Classes," reflecting its evolution from the C programming language. The introduction of classes represented an extension of the structures (struts) found in C. While structures in C had limitations—such as having all members public and lacking methods—C++ addressed these shortcomings by

allowing the creation of classes, which combine the features of structures with additional capabilities. Classes in C++ can have both methods and properties, enabling encapsulation and abstraction. They also provide flexibility in access control, allowing some members to be declared as private while others can remain public. Furthermore, C++ permits the declaration of objects alongside the class definition, as illustrated in the example with the `Student` class, where instances like `Haseeb`, `Bashir`, and `Salaar` can be declared directly within the class declaration. This powerful combination of features makes C++ a robust language for objectoriented programming.

```
#include<iostream> using
namespace std;
class Student{
private:
    int a , b ,c;
public:
    int d , e ;          void setData(int a,
int b, int c);          void getData(){
cout<<"The value of a is "<<a<<endl;
cout<<"The value of b is "<<b<<endl;
cout<<"The value of c is "<<c<<endl;
cout<<"The value of d is "<<d<<endl;
cout<<"The value of e is "<<e<<endl;
    }
};
// :: scope resolution operator
void Student:: setData(int a1 , int b1 , int c1){
a  = a1;      b  = b1;      c  = c1;
}
int main(){
Student Sahoo;
    Sahoo.d = 34;
    Sahoo.e = 12;
    Sahoo.setData(1,3,5) ;
Sahoo.getData();      return
0;
}
```

Access Specifiers

In C++, **access specifiers** (or access modifiers) determine the visibility and accessibility of class members (attributes and methods) to other parts of the program. There are three main access specifiers in C++: **public**, **protected**, and **private**. Each serves a distinct purpose in controlling how class members can be accessed:

Public

Members declared as **public** are accessible from anywhere in the program. This means that any code can access and modify public members of a class, making them suitable for methods and attributes that need to be widely accessible.

```
class Person { public:
    string name; // Public member
void display() { // Public method
cout << "Name: " << name << endl;
    }
};

Person p;
p.name = "Alice"; // Accessible
p.display(); // Accessible
```

Private

Members declared as **private** are only accessible within the class itself. They cannot be accessed directly from outside the class, which helps encapsulate data and protect it from unintended modifications. This is useful for attributes that should not be exposed to the outside world. By default each member is private.

```

class BankAccount { private:
    double balance; // Private member
public:
    void deposit(double amount) { // Public method
balance += amount; // Can access private member
    }
    double getBalance() { // Public method
        return balance; // Can access private member
    }
};

BankAccount account;
// account.balance = 1000; // Error: 'balance' is
private account.deposit(1000); // Accessible cout <<
account.getBalance(); // Accessible

```

Protected:-

Members declared as **protected** are accessible within the class itself and by derived classes (subclasses), but not by other code outside the class hierarchy. This allows for controlled access in inheritance scenarios, making it useful when you want derived classes to have access to certain attributes or methods while keeping them hidden from the rest of the program.

```

class Base { protected:
    int protectedValue; // Protected member
};

class Derived : public Base { public:
    void setProtectedValue(int value) {
protectedValue = value; // Accessible in derived class
    }
};

Derived d;
// d.protectedValue = 10; // Error: 'protectedValue' is protected
d.setProtectedValue(10); // Accessible through public method

```

Encapsulation

Encapsulation is one of the four fundamental principles of **object-oriented programming (OOP)**, along with **inheritance**, **polymorphism**, and **abstraction**. It refers to the bundling of data (variables) and methods (functions) that operate on the data into a single unit or class, while restricting the access to some of the object's components to prevent accidental modification or misuse.

In simple terms, **encapsulation** is about hiding the internal state of an object and allowing access to it through well-defined interfaces, typically using **getters** and **setters**.

Key Features of Encapsulation:

1. **Data Hiding:** The internal data of an object is hidden from the outside world. Only specific methods (such as getters and setters) can access or modify the data, ensuring control over how the data is accessed or modified.
2. **Controlled Access:** Encapsulation allows us to control how data is accessed by limiting direct access to some components of an object. This prevents unintended interference or corruption of data.

```

#include<iostream>

using namespace std;
class Employee
{ private:
    // Private data members (not accessible outside the
class)    int id;    string name;

public:
    // Public member function to set the private data    void
setData(int empId, string empName) {        id = empId;    // Assigning
the passed value to the private member        name = empName;
    }

    // Public member function to get the data
void displayData() {        cout << "Employee
ID: " << id << endl;        cout << "Employee
Name: " << name << endl;
    }
};

int main() {
    Employee emp;

    // Set data using the public setter method
emp.setData(101, "John Doe");

    // Access data using the public getter method
emp.displayData();

    return 0;
}

```

Advantages of Encapsulation:

1. **Data Protection:** Encapsulation provides security by restricting direct access to the data and only allowing controlled access via public methods.
2. **Code Maintainability:** Since the data is hidden, changes to the internal implementation of a class will not affect other parts of the program that use the class. You can modify the internal implementation without affecting external code.
3. **Improved Flexibility:** Encapsulation allows adding validation logic inside getter/setter methods before accessing or modifying the data.

4. **Increased Reusability:** Encapsulated classes are more modular and easier to reuse in different parts of a program or in different programs.

Nesting of Member Functions

In C++, **nesting of member functions** refers to the practice of calling one member function from another within the same class. This can enhance code organization, improve readability, and allow for better management of class behaviors. Nesting functions is especially useful when a member function performs a task that is logically a part of another function's operation, facilitating code reuse and reducing redundancy.

```
#include <iostream> using
namespace std;

class Calculator { public:
    // Member function to add two
    numbers    int add(int a, int b) {
return a + b; // Returns the sum
    }

    // Member function to calculate the square of the sum of two
    numbers    int squareOfSum(int x, int y) {        int sum = add(x,
y); // Nesting: calling add() inside squareOfSum()        return sum *
sum; // Returning the square of the sum
    }
};

int main() {
    Calculator calc; // Creating an object of the Calculator class
    int result = calc.squareOfSum(3, 4); // Calling squareOfSum()    cout <<
"The square of the sum is: " << result << endl; // Output: 49    return
0;
}
```

In above example we are calling the **add** function from the **squareOfSum** function within the same class **Calculator**.

Memory Allocation

In C++, memory allocation for classes and objects can be handled using both **static** and **dynamic** allocation. Understanding how memory is allocated and how to use arrays of objects is crucial for effective programming in C++. Here's an overview of these concepts:

Static Memory Allocation:-

- This occurs when the memory for objects is allocated at compile time.
- Objects are created with automatic storage duration (e.g., local objects in functions) or static storage duration (e.g., global objects or static local objects).
- The lifetime of these objects is managed by the compiler.

```
class Box { public:
    int length;
    int width;    int
height;
};
int main() {
    Box box1; // Static
allocation    box1.length = 10;
box1.width = 5;    box1.height =
2;
    // Memory is automatically deallocated when box1 goes out of scope
return 0;
}
```

Dynamic Memory Allocation:-

- This occurs when the memory for objects is allocated at runtime using the `new` operator.
- This allows for the creation of objects whose size can be determined at runtime.
- It is important to release dynamically allocated memory using the `delete` operator to prevent memory leaks.

```
class Box { public:
```

```

    int length;
int width;    int
height;
};
int main() {
    Box* box2 = new Box(); // Dynamic allocation
box2->length = 10;    box2->width = 5;
box2->height = 2;    delete box2; // Freeing
memory    return 0;
}

```

Arrays of objects

In C++, you can create arrays of objects both statically and dynamically.

Static Array of Objects:-

You can declare an array of objects at compile time, which allocates memory for all objects at once.

```

class Box { public:
    int length;
int width;    int
height;
};
int main() {
    Box boxes[3]; // Static array of 3 Box
objects    boxes[0].length = 10;
boxes[1].width = 5;    boxes[2].height = 2;
    // All objects are automatically destroyed when the program exits the
scope
    return 0;
}

```

Dynamic Array of Objects:-

To create a dynamic array of objects, you allocate memory for the array using the `new` operator and can specify the size at runtime.

```

class Box { public:
    int length;
    int width;    int
height;
};
int main() {    int n = 3; // Size
determined at runtime
    Box* boxes = new Box[n]; // Dynamic array of Box objects

    for (int i = 0; i < n; i++) {        boxes[i].length = i
+ 1; // Setting length for each Box        boxes[i].width = i
+ 2; // Setting width for each Box        boxes[i].height =
i + 3; // Setting height for each Box    }

    // Memory cleanup
    delete[] boxes; // Freeing the dynamically allocated array
return 0;
}

```

In summary, C++ provides both static and dynamic memory allocation for classes and objects. Static allocation is handled at compile time, while dynamic allocation allows for greater flexibility during runtime. Arrays of objects can be created statically or dynamically, providing a structured way to manage multiple instances of a class. When using dynamic memory allocation, always ensure to release memory properly to avoid memory leaks. Understanding these concepts is crucial for effective memory management in C++.

Static Methods and Members

In C++, **static members** and **static methods** are class members that belong to the class itself rather than to any specific object instance. This means they can be accessed without creating an instance of the class. Here's a detailed overview of static members and methods:

Static Data Members:-

□ A static data member is shared among all instances of a class. It means that there is only one copy of the static member for the entire class, regardless of how many objects are

created. □ Static data members are initialized outside the class definition, typically in a source file, and they must be defined with the keyword `static`.

Static Member Functions:-

- A static member function can only access static data members and other static member functions. It cannot access non-static members because it does not operate on an instance of the class.
- Static methods can be called without creating an instance of the class.

```
#include <iostream> using
namespace std;

class Counter { private:
    static int count; // Static member variable to keep track of object
count public:
    // Static method to increment the
count    static void increment() {
count++;
    }
    // Static method to get the current
count    static int getCount() {
return count;
    }
};

// Definition of the static member variable int
Counter::count = 0;

int main() {
    // Incrementing the count without creating any instances of Counter
    Counter::increment(); // Increment count to 1
    Counter::increment(); // Increment count to 2

    cout << "Number of objects created: " << Counter::getCount() << endl; //
// Output: 2

    // Further increments can be done without any objects
    Counter::increment(); // Increment count to 3

    cout << "Number of objects created: " << Counter::getCount() << endl; //
Output: 3

    return 0;
}
```

Friend Function

A **friend function** in C++ is a function that is not a member of a class but is given permission to access the private and protected members of the class. Normally, private members of a class are not accessible from outside the class, but the friend function breaks this rule by being granted special access.

Key Characteristics of a Friend Function

❑ Non-Member Function:

- A friend function is not part of the class in which it is declared. It is a regular function that resides outside the class.
- However, it is given access to the class's private and protected members.
- **Declared Using `friend` Keyword:**
- A function becomes a friend of a class when it is declared inside the class using the keyword `friend`.
- The declaration inside the class only grants access; the function is still defined outside the class, just like a regular function.
- It can be declared in public and also in private section
- **Access to Private and Protected Members:**
- The primary advantage of a friend function is that it can access private and protected members of the class directly.
- It allows the friend function to manipulate the internal state of the class.
- **Not Invoked Using Object:-**

Unlike member functions, a friend function is called like a normal function without using the object's dot operator (.). Instead, it takes objects of the class as arguments.

```
#include<iostream>
using namespace std;
class Complex{
int a , b ;
```

```

    public:
        void setNumber(int n1 , int
n2){          a = n1;          b = n2;
        }
        friend Complex sumComplex(Complex o1 , Complex
o2);
        void printNumber(){          cout<<"Your number
is "<<a<<" + "<<b<<"i"<<endl;
        }
};
Complex sumComplex(Complex o1 , Complex o2){
Complex o3;    o3.setNumber((o1.a + o2.a) ,
(o1.b + o2.b) );    return o3;
}
int main(){
    Complex c1 ,c2 , sum;
c1.setNumber(3,4);
c1.printNumber();
c2.setNumber(6,7);
c2.printNumber();    sum =
sumComplex(c1 , c2);
sum.printNumber();    return
0;
}

```

Syntax:-

```

class ClassName {
    // Private members
private:
    int privateData;

    // Friend function declaration    friend
void friendFunction(ClassName obj);
};

```

When to Use a Friend Function?

- **Operator Overloading:** Friend functions are commonly used to overload certain operators that require access to private members of two different objects.

- **Inter-Class Functionality:** When you want to implement functionality that involves multiple classes and you need access to their internal details.
- **Non-Member Function:** When a function logically doesn't belong to a class but still needs to access its private members.

Friend function of another class

```
#include<iostream>
using namespace std;

//Forward
declaration class
Complex; class
Calculator{
public:
    int sumRealComplex(Complex , Complex );
}; class Complex{    int a , b ;    friend int Calculator
::sumRealComplex(Complex, Complex );    public:
    void setNumber(int n1 , int
n2){        a = n1;        b = n2;
    }
    void printNumber(){        cout<<"Your number
is "<<a<<" + "<<b<<"i"<<endl;
    }
}; int Calculator :: sumRealComplex(Complex obj1, Complex
obj2){        return  obj1.a + obj2.a;
}

int main(){
    Complex o1 , o2 ;
o1.setNumber(1,4);
o2.setNumber(5,7);

    Calculator cal;
```

```

        cout<<cal.sumRealComplex(o1,o2); //will return 6
    return 0;
}

```

□

Friend Class

In C++, a **friend class** is a class that has access to the private and protected members of another class. Declaring a friend class means that all the member functions of the friend class can access the private and protected members of the class that declares the friendship.

```

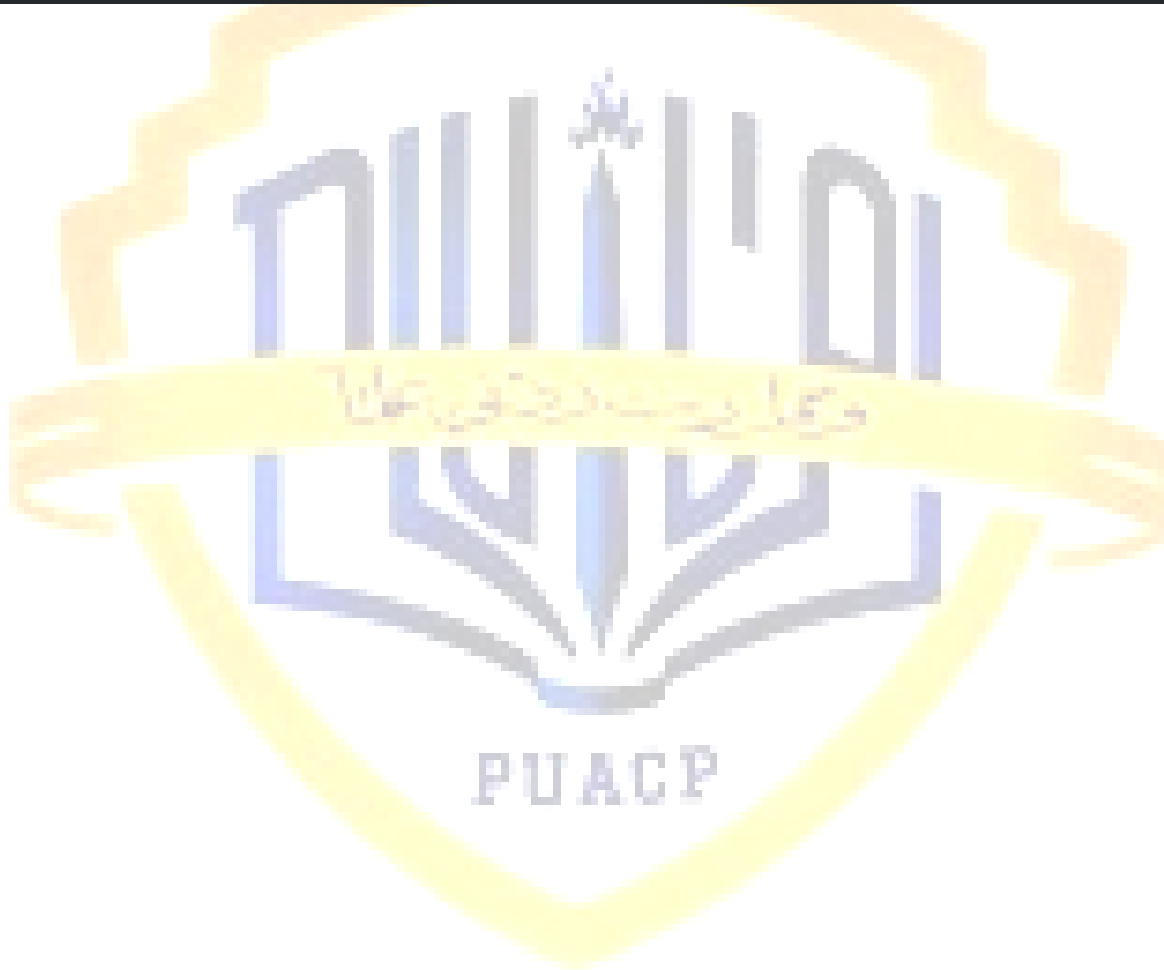
#include<iostream>
using namespace std;

//Forward
declaration class
Complex; class
Calculator{
public:
    int sumRealComplex(Complex , Complex );
int sumImgComplex(Complex , Complex );
}; class Complex{    int
a , b ;    friend class
Calculator;    public:
    void setNumber(int n1 , int
n2){        a = n1;        b = n2;
    }

    void printNumber(){
        cout<<"Your number is "<<a<<" + "<<b<<"i"<<endl;
    }
}; int Calculator :: sumRealComplex(Complex obj1, Complex
obj2){        return  obj1.a + obj2.a;
} int Calculator :: sumImgComplex(Complex obj1, Complex
obj2){        return  obj1.b + obj2.b;
} int
main(){

```

```
    Complex o1 , o2 ;  
o1.setNumber(1,4);  
o2.setNumber(5,7);  
  
    Calculator cal;  
cout<<cal.sumRealComplex(o1,o2)<<endl;  
cout<<cal.sumImgComplex(o1,o2)<<endl;  
return 0;  
}
```



Example:-

```
#include<iostream>
using namespace std;

class Y;
class X{    int data ;
friend void add(X , Y);
public:
    void setValue(int value){
data = value ;
    }
}; class Y{    int num ;
friend void add(X , Y);
public:
    void setValue(int value){
num = value;
    }
}; void add(X o1 , Y
o2){
    cout<<"Sum of datas of X and Y is "<<o1.data + o2.num<<endl;
}
int main(){
X a;
    a.setValue(4);
Y b;
    b.setValue(5);

    add(a,b);
return 0; }
```

Constructors

In C++, **constructors** are special member functions used to initialize objects of a class. They are automatically called when an object of a class is created. A constructor has the same name as the class and does not have a return type, not even `void`.

```
#include<iostream>
using namespace std;
class Complex{ public:    Complex(){
cout<<"Hello World, i am constructor"<<endl;
}
};
int main(){
Complex c;
return 0;
}
```

Types of Constructors

1. **Default Constructor:** A constructor with no parameters.
2. **Parameterized Constructor:** A constructor that takes parameters to initialize the object.
3. **Copy Constructor:** A constructor that creates a new object as a copy of an existing object.

Default Constructure:-

A **default constructor** is a constructor that takes no arguments or has default arguments. It is automatically called when an object is created without passing any parameters.

```
class ClassName { public:
    ClassName() {
        // Initialization code
    }
};
```

Parameterized Constructor:-

A **parameterized constructor** is a constructor that takes one or more parameters to initialize an object with specific values when it is created.

```
class ClassName { public:
    ClassName(int param1, int param2) {
        // Use the parameters for initialization
    }
};
```

Copy Constructor:-

A **copy constructor** is a constructor that creates a new object as a copy of an existing object. It takes a reference to an object of the same class as its parameter.

```
class ClassName { public:
    ClassName(const ClassName &obj) {
        // Copy initialization
    }
};
```

Constructors Overloading

Constructor overloading in C++ refers to the ability to define multiple constructors with different parameter lists for the same class. This allows objects of a class to be initialized in different ways depending on how many arguments (or types of arguments) are passed during object creation.

Key Points:

- **Multiple Constructors:** You can have more than one constructor, each with a different set of parameters.
- **Automatic Selection:** When you create an object, C++ automatically chooses the appropriate constructor based on the number and types of arguments provided.
- **Default Constructor:** If no parameters are passed, the default constructor is chosen.
- **Parameterized Constructor:** If parameters are passed, the constructor with matching parameters is chosen.

```
class ClassName { public:
    ClassName() {
        // Default constructor
    }
```

```
    ClassName(int param) {
        // Constructor with one integer parameter
    }
```

```

    }

    ClassName(int param1, double param2) {
        // Constructor with an integer and a double parameter
    }
};

```

Constructors with default arguments

In C++, constructors can have **default arguments**, which provide default values for parameters if no arguments are passed when an object is created. This allows flexibility in object initialization without the need for multiple constructor overloads. When a constructor has default arguments, it can be used with or without passing parameters.

```

class ClassName { public:
    ClassName(int param1 = 0, int param2 = 0) {
        // Constructor body
    }
};

```

Key Points:

- Default arguments in constructors allow you to specify default values for parameters, making constructor calls more flexible.
- You can pass some or all arguments when creating an object; if an argument is not provided, the default value is used.
- This feature can be used to simplify constructor overloading by covering multiple use cases with a single constructor that has default arguments.

Dynamic initialization of objects using Constructors

Dynamic initialization refers to initializing an object with values that are not known at compile time but are instead provided at runtime (e.g., through user input). In the context of constructors, this means the object's state can be determined when it is created.

```

#include <iostream> using
namespace std;
class BankDeposit {
int principal;    int
year;    float
interestRate;
float returnValue;
public:
    BankDeposit() {} // Default constructor
    BankDeposit(int p, int y, float r); // Constructor with float interest rate
    BankDeposit(int p, int y, int r);    // Constructor with int interest rate
    void show(); // Function to display
results
};

// Constructor to initialize using float interest
rate BankDeposit::BankDeposit(int p, int y, float r)
{
    principal = p;    year = y;
    interestRate = r;
    returnValue = principal; // Initialize returnValue to principal
    for (int i = 0; i < year; i++) {        returnValue *= (1 + interestRate);
// Compound interest calculation
    }
}

// Constructor to initialize using int interest
rate BankDeposit::BankDeposit(int p, int y, int r)
{
    principal = p;    year = y;
    interestRate = float(r) / 100; // Convert percentage to decimal
    returnValue = principal; // Initialize returnValue to
principal    for (int i = 0; i < year; i++) {
        returnValue *= (1 + interestRate); // Compound interest calculation
    }
}

// Function to display the principal and return value
void BankDeposit::show() {    cout << "Principal amount
was: " << principal << endl

```

```

        << "Return value after " << year << " years is: " << returnValue <<
endl;
    }
int main() {
    BankDeposit c; // Default constructor
called    int p, y, R;    float r;
    cout << "Enter the values of principal (p), years (y), and interest rate
(r):
" << endl;    cin >> p
>> y >> r;
    c = BankDeposit(p, y,
r);
    c.show(); // Display the results
return 0;
}

```

Explanation of the Code

1. Class Definition:

- The `BankDeposit` class has private members for principal, year, interestRate, and returnValue.
- There are three constructors: a default constructor and two parameterized constructors (one for float interest rate and one for integer interest rate).

2. Parameterized Constructors:

- The first constructor takes parameters for principal, years, and a float interest rate.
- The second constructor takes parameters for principal, years, and an integer interest rate, converting the integer to a float by dividing by 100.

3. Dynamic Initialization:

- In the `main()` function, the user is prompted to enter values for the principal amount, the number of years, and the interest rate.
- The statement `c = BankDeposit(p, y, r);` dynamically initializes the `BankDeposit` object `c` using the provided input. This uses the appropriate constructor based on the type of the interest rate.

4. Interest Calculation:

- In both constructors, the compound interest formula is applied over the specified number of years to calculate the return value.

5. Display Method:

- The `show()` method outputs the principal amount and the return value after the specified number of years.

Copy Constructor²¹

A **copy constructor** in C++ is a special constructor that creates a new object as a copy of an existing object. It is called when a new object is initialized from an existing object (e.g., when passing an object by value or returning an object from a function). By default, C++ provides a **default copy constructor**, which performs a shallow copy. However, you can define your own copy constructor to perform a **deep copy** if necessary.

```
class ClassName { public:  
    ClassName(ClassName &obj); // Copy constructor  
};
```



Example:-

```
#include<iostream> using namespace std;
class Number{    int a;    public:
Number(){        a  = 0;
    };
    Number(int num){        a = num;
    }

    Number(Number &obj){        cout<<"Copy constructor
called"<<endl;        a = obj.a;
    }
    void display(){
        cout<<"The number for this object is "<<a<<endl;
    }
};

int main(){    Number x,y,z , z1;    x =
Number(4);
    x.display();
    y.display();
    z.display();

    z1 =
Number(x);
z1.display();
    // z1 should exactly resemble z or x or y
return 0;
}
```

Key Points:

- A **copy constructor** creates a new object as a copy of an existing object.
- The **default copy constructor** performs a shallow copy (bitwise copy of members).
- A **deep copy constructor** is needed when objects contain pointers or dynamic resources to ensure independent copies.
- The copy constructor is invoked when:

- An object is passed by value. ○ An object is returned by value.
- An object is explicitly copied during initialization or assignment.

```
int main(){
    Number x, y, z , z1 , z2;
    x = Number(4);

    z2 = x ; //will not invoked
    Number z3 = x ; // will invoked

    return 0;
}
```

Destructor

In C++, a **destructor** is a special member function that is automatically invoked when an object goes out of scope or is explicitly deleted. Its main purpose is to free resources that the object may have acquired during its lifetime, such as memory, file handles, or network connections. Destructors are used to ensure that no memory leaks or other resource issues occur.

Key Characteristics of Destructors:

1. **Name:** The destructor has the same name as the class but is prefixed with a tilde ~. For example, if the class is `MyClass`, the destructor will be `~MyClass()`.
2. **No Arguments:** A destructor cannot take parameters and does not return any value.
3. **Automatically Called:** The destructor is called automatically when the object's lifetime ends, so you don't have to call it manually.
4. **Single Destructor:** A class can only have one destructor, and it cannot be overloaded.
5. **Destructor for Inheritance:** When a class is derived from another, the destructors are called in the reverse order of the constructor calls, starting with the derived class and then calling the base class destructor.

```
class MyClass {
public:
    // Constructor
    MyClass() {
        // Initialization code
        cout <<
        "Constructor called!" << endl;
    }

    // Destructor
}
```

```

~MyClass() {           // Cleanup code
cout << "Destructor called!" << endl;
}
};

```

Use Cases for Destructors:

- **Memory Management:** Free dynamically allocated memory using `delete` or `delete[]`.
- **Closing Files or Connections:** Close file handles or database connections to avoid resource leaks.
- **Releasing Locks:** Release any locks or synchronization mechanisms to ensure smooth multithreaded operation.

If you're working with dynamic memory allocation (e.g., with `new`), the destructor becomes critical for preventing memory leaks by ensuring the allocated memory is properly released.

```

#include<iostream>
using namespace std;
int count =
0;
class num{    public:    num(){        count++;
cout<<"Constructor is called when count is "<<count<<endl;

    }

    ~num(){        count--;        cout<<"Destructor
is called and cou t is "<<count<<endl;
    }
}; int
main(){
num a;
return 0;
}

```

Dynamically memory example:-

```
#include <iostream> using
namespace std;
class MyClass
{ private:
    int* data; // Pointer to dynamically allocated memory public:
    // Constructor: Dynamically allocates
memory    MyClass(int value) {        data =
new int; // Allocate memory
        *data = value; // Assign value to allocated memory        cout <<
"Constructor: Memory allocated and value set to " << *data << endl;    }

    // Destructor: Frees dynamically allocated memory
    ~MyClass() {
        delete data; // Free the allocated memory
cout << "Destructor: Memory deallocated" << endl;    }

    // A function to get the
value    int getValue() {
return *data;
    }
};

int main() {
    MyClass obj(42); // Constructor is called, memory allocated, value set
cout << "Value: " << obj.getValue() << endl;

    // Destructor is called automatically when obj goes out of scope
return 0;
}
```

Inheritance

In C++, **inheritance** is one of the core principles of object-oriented programming (OOP). It allows a class (called a **derived class**) to inherit attributes and methods from another class

(called a **base class**). This promotes code reusability and establishes a hierarchical relationship between classes.

```
class BaseClass {  
    // Base class members  
};  
class DerivedClass : accessSpecifier BaseClass {  
    // Derived class members  
};
```

Key Points:

- The derived class can use all **public** and **protected** members of the base class but cannot directly access **private** members of the base class.
- The derived class can add its own members (attributes and methods).
- The derived class can override or extend the functionality of the base class by redefining base class methods.

Access Control in Inheritance:

1. **Public Inheritance:**

- Public members of the base class remain public in the derived class.
- Protected members of the base class remain protected in the derived class.
- Private members of the base class remain inaccessible to the derived class.

2. **Protected Inheritance:**

- Public and protected members of the base class become protected in the derived class.

3. **Private Inheritance:**

- Public and protected members of the base class become private in the derived class.

Types of Inheritance:

1. **Single Inheritance:** A class inherits from one base class.

```

#include <iostream> using
namespace std;

// Base class
class Animal {
public:
    void eat() {          cout << "I
can eat!" << endl;
    }
};

// Derived class
class Dog
: public Animal { public:
    void bark() {          cout << "I
can bark!" << endl;
    }
};

int main() {
    Dog myDog;
    // Calling methods from base class
myDog.eat(); // Inherited from Animal
// Calling method from derived class
myDog.bark(); // Defined in Dog    return
0;
}

```

- **Multiple Inheritance:** Multiple inheritance allows a derived class to inherit from more than one base class. This can lead to ambiguity if both base classes have methods with the same name.

```

class Base1 {
public:
    void display() {
        cout << "Base1 class display" << endl;
    }
};
class
Base2 {
public:

```

```

    void show() {          cout << "Base2
class show" << endl;
    }
}; class Derived : public Base1, public
Base2 {
};
int main() {      Derived obj;
obj.display();    // Inherited from Base1
obj.show();       // Inherited from Base2
}

```

- **Multilevel Inheritance:** In multilevel inheritance, a class is derived from a class which is also derived from another class, forming a chain of inheritance.

```

class Base {
public:
    void baseDisplay() {
cout << "Base class" << endl;
    }
}; class Intermediate : public
Base { public:
    void intermediateDisplay() {
cout << "Intermediate class" << endl;
    } };
class Derived : public Intermediate { public:
    void derivedDisplay() {
cout << "Derived class" << endl;
    }
};
int main() {      Derived obj;      obj.baseDisplay();          //
Inherited from Base class      obj.intermediateDisplay(); //
Inherited from Intermediate class      obj.derivedDisplay();    //
Defined in Derived class }

```

- **Hierarchical Inheritance:** In hierarchical inheritance, multiple classes are derived from a single base class. Each derived class can add its own functionality, while still inheriting from the base class.

```
class Base {
public:    void
display() {

        cout << "Base class display" << endl;
    } }; class Derived1 :
public Base { public:
        void show() {          cout << "Derived1
class show" << endl;
        } }; class Derived2 :
public Base { public:
        void show() {          cout << "Derived2
class show" << endl;
        } };
int main() {      Derived1 obj1;      Derived2
obj2;      obj1.display(); // Inherited from
Base class      obj1.show();      // Defined in
Derived1      obj2.display(); // Inherited from
Base class      obj2.show();      // Defined in
Derived2 }
```

- **Hybrid Inheritance:** Hybrid inheritance is a combination of two or more types of inheritance. It often includes the use of multiple and hierarchical inheritance, but care must be taken to avoid ambiguity in such designs. In some cases, virtual base classes are used to resolve conflicts.

```
class Base {
public:
        void display() {
            cout << "Base class" << endl;
        }
};
class Derived1 : public Base {
};
```

```

class Derived2 : public Base {
}; class FinalDerived : public Derived1, public Derived2
{ };
int main() {
    FinalDerived obj;
    // obj.display(); // Error: Ambiguity, both Derived1 and Derived2 inherit from
Base
}

```

Summary:

1. **Single Inheritance:** One class inherits from one base class.
2. **Multiple Inheritance:** One class inherits from multiple base classes.
3. **Multilevel Inheritance:** A class inherits from another derived class, forming a chain.
4. **Hierarchical Inheritance:** Multiple classes inherit from the same base class.
5. **Hybrid Inheritance:** A combination of multiple inheritance types.

Each type of inheritance has its specific use cases, and care must be taken, especially with **multiple inheritance**, to avoid ambiguity. Virtual inheritance can be used to solve conflicts arising from such cases.

Exercise

Create 2 classes:

1. SimpleCalculator – Take input of 2 numbers using a utility function and performs +, -, *, / and displays the results using another function.
2. ScientificCalculator – Take input of 2 numbers using utility function and performs any four scientific operation of your choice and displays the results using another function.

Create another class HybridCalculator and inherit it using these 2 classes, also answer below questions:

Q1: What type of inheritance are you using ?

Q2: Which mode of inheritance are you using?

Q3: Create an object of HybridCalculator and display results of simple and scientific calculator.

Ambiguity Resolution

In **multiple inheritance**, ambiguity arises when two or more base classes have member functions (or data members) with the same name, and a derived class inherits from these base classes. When you try to access that member from the derived class, the compiler cannot determine which base class's member should be used, causing an **ambiguity error**.

Example Case:-

```
#include<iostream>
using namespace std;
class Base1
{ public:
    void display() {          cout <<
"Display from Base1" << endl;
    } };
class Base2
{ public:
    void display() {          cout <<
"Display from Base2" << endl;
    } }; class Derived : public Base1,
public Base2 {
};
int main() {      Derived obj;      obj.display(); // Ambiguity Error: Which
display() should be called, Base1 or Base2?
    return 0;
}
```

Error:

When you try to call `obj.display()`, the compiler gives an error because it cannot decide whether to call `Base1::display()` or `Base2::display()`.

Ambiguity Resolution:

There are several ways to resolve such ambiguities in C++.

1. Scope Resolution Operator (::):

You can specify which base class's member function you want to call by using the scope resolution operator.

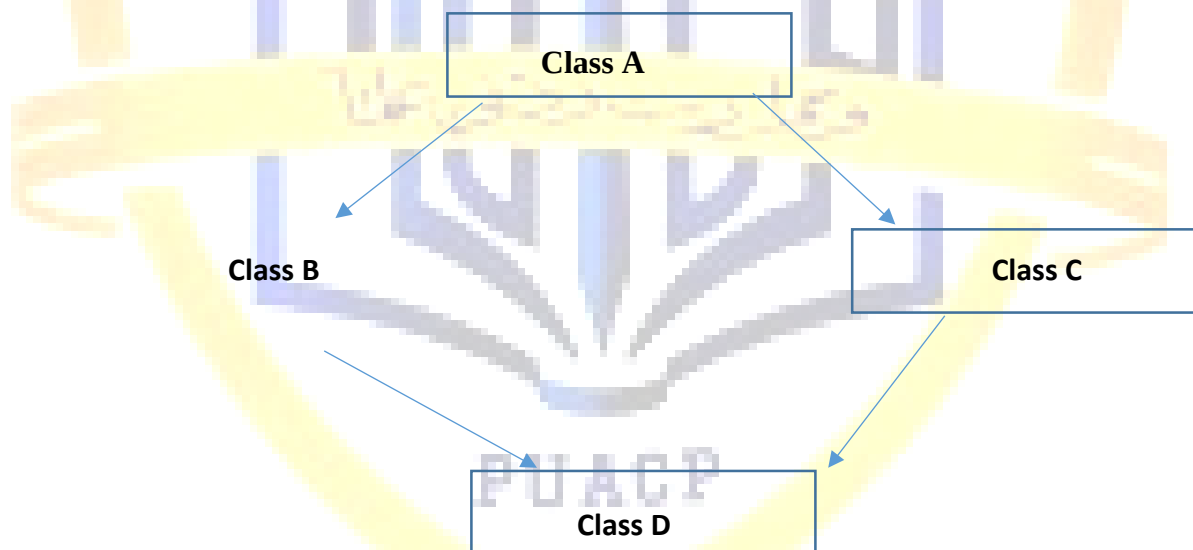
```
int main() {    Derived obj;
obj.Base1::display(); // Calls Base1's
display()    obj.Base2::display(); //
Calls Base2's display()    return 0;
}
```

Note:-

- If there is also same function in derived class then function will be override and compiler will run the derived class function without ambiguity.

2. Virtual Base Classes:

Ambiguity can also arise in the context of **diamond inheritance**, where two base classes inherit from a common base class, and the derived class inherits from both base classes. In such cases, **virtual inheritance** is used to avoid duplication of the common base class and resolve ambiguity.



In the above diagram, if we declare an `int` variable `number` inside class A, it will be inherited by both class B and class C. Then, class D will inherit the `number` from both class B and class C. Then compiler confuse what should it do?

A **virtual base class** in C++ is used to solve the **diamond problem** or **ambiguity problem** that arises in multiple inheritance. When a derived class inherits from two or more base classes, which in turn inherit from a common base class, the derived class ends up with **multiple copies** of the common base class. This can lead to ambiguity and unnecessary duplication of data members. Using **virtual inheritance** ensures that only **one copy** of the base class is inherited, avoiding duplication and ambiguity.

```

#include<iostream> using
namespace std;
class A
{
public:
    int number;
}; class B : virtual public A
{ };
class C : virtual public A {
};
class D : public B, public C {
};

int main() {    D obj;    obj.number = 10; // Now there is
only one 'number' from A
    cout << "Number: " << obj.number << endl; // No ambiguity
    return
0;
}

```

Const vs Non const Functions

In C++, the `const` keyword can be used to indicate that a member function will **not modify the object** on which it is called. This distinction is crucial in object-oriented programming as it helps maintain the immutability of certain operations and ensures that some methods do not accidentally alter the object's state.

Non-const Functions:-

- **Non-const** member functions can modify the state (data members) of the object on which they are called.
- These are the regular member functions in C++.
- Any function that is not explicitly marked as `const` is considered non-const by default.

```

#include<iostream>
using namespace std;
class MyClass
{ private:
    int value;
public:
    MyClass(int v) : value(v) {}

    // Non-const function    void setValue(int v) {
value = v; // Modifies the member variable 'value'    }
void display() {          cout << "Value: " << value <<
endl;
    }
};
int main() {
    MyClass obj(10);    obj.display();
// Output: Value: 10
    obj.setValue(20); // Modifies the
value    obj.display(); // Output: Value:
20
    return
0;
}

```

Const Functions:-

- **const** member functions **cannot modify** any of the member variables of the class (unless those members are explicitly declared as `mutable`).
- A `const` function guarantees that it will not change the object's state when called.
- These functions are mainly used when you want to perform **read-only** operations on the object without altering its state.
- To declare a `const` function, the `const` keyword is added **after the parameter list** of the function.
- `const` member functions can be called on both **const** and **non-const** objects, while non-const functions can only be called on **non-const** objects.

```

#include<iostream>
using namespace std;
class MyClass
{

```

```

private:
    int value;
public:
    MyClass(int v) : value(v) {}

    // Const function that does not modify the member variables
int getValue() const { return value;
} void display() const {
cout << "Value: " << value << endl;
}

    // Non-const function void setValue(int v) {
value = v; // Modifies the member variable 'value'
}
}; int main() { const MyClass obj(10); // 'obj' is
a const object
    obj.display(); // This works because display() is a const function //
obj.setValue(20); // ERROR: Cannot call non-const function on a const object
return 0;
}

```

PUACP

Initialization list in constructors

In C++, an **initialization list** is a special syntax used to initialize member variables and base class constructors before the body of the constructor is executed. It is particularly useful when initializing **const members**, **reference members**, or **class members** that do not have default constructors. It also ensures that base class constructors are called with the right arguments.

```

class ClassName { private:    int x;    int y;
public:

```

```

ClassName(int a, int b) :
x(a), y(b) { // Initialization
list
    // Constructor body
}
};

```

Key Reasons to Use Initialization Lists

1. **Initialization of Constant Data Members (`const`)** ◦ `const` members must be initialized at the time of object creation and cannot be modified later.

```

class MyClass {
private:
    const int x;
public:
    MyClass(int val) : x(val) { // Must use initialization list
        // Can't assign a value to 'x' here
    }
};

```

Initialization of Reference Data Members (&)

- References must be initialized when they are declared, so an initialization list is the only way to initialize them.

```

class MyClass {
private:
    int& ref;
public:
    MyClass(int& r) : ref(r) { // Initialize reference with 'r'
    }
};

```

Initialization of Members That Don't Have Default Constructors

- If a class member does not have a default constructor, it must be initialized via an initialization list.

```

class MemberClass {
public:
    MemberClass(int x) {

```

```

        std::cout << "MemberClass constructor with value: " << x << std::endl;
    }
}; class
MyClass {
private:
    MemberClass member;
public:
    MyClass(int val) : member(val) { // Use initialization list to
call MemberClass constructor      std::cout << "MyClass constructor"
<< std::endl;
    }
};

```

Better Performance

- Using an initialization list is more efficient than assigning values in the constructor body because members are initialized directly instead of being initialized with default values and then reassigned.

Example: Initialization List with Base Class

When dealing with inheritance, the initialization list is also used to initialize the base class constructor.

```

class Base {
public:
    Base(int x) {          std::cout << "Base class constructor called with
value: " << x << std::endl;
    }
}; class Derived : public
Base { private:    int y;
public:
    Derived(int x, int yVal) : Base(x), y(yVal) { // Base class initialized here
std::cout << "Derived class constructor called with value: " << y << std::endl;
    }
};

```

Advantages of Initialization Lists

- **Direct Initialization:** Prevents default initialization followed by assignment.
- **Mandatory for Certain Members:** `const`, reference, and non-default-constructed members.
- **Cleaner Code:** The intention to initialize members is explicit.

Pointers

In C++, pointers and dynamic memory management are fundamental concepts, and the `new` and `delete` keywords are used for **dynamic memory allocation** and **deallocation**.

1. The `new` Keyword (Dynamic Memory Allocation)

The `new` keyword allocates memory on the **heap** (also known as free store) and returns a pointer to the memory location. This allows the program to request memory dynamically at runtime rather than at compile time (which is the case with statically allocated memory).

Syntax:

```
int* ptr = new int;
```

The above code:

- Allocates memory for an `int` on the heap.
- Returns a pointer (`ptr`) to the allocated memory.

Allocating Arrays Dynamically:

You can also allocate an array dynamically using `new`:

```
int* arr = new int[5];
```

Example:

```
int* ptr = new int(42); // Dynamically allocate memory for an int with value 42
cout << "Value: " << *ptr << endl; // Dereference to access the value
delete ptr; // Free the allocated
memory ptr = nullptr;
```

2. The `delete` Keyword (Dynamic Memory Deallocation)

When memory is no longer needed, it should be deallocated using the `delete` keyword to prevent **memory leaks** (i.e., memory that is allocated but never released).

```
delete ptr;
```

After calling `delete`, the pointer (`ptr`) becomes a **dangling pointer** (i.e., it points to deallocated memory), so it's good practice to set it to `nullptr` to avoid accessing invalid memory.

Deleting Arrays:

When you allocate an array dynamically, you must use the array version of `delete`, called `delete[]`.

```
int* arr = new int[5];  
delete[] arr;
```

Important Points:

1. `new` allocates memory on the heap, and `delete` frees that memory.
2. For every call to `new`, there should be a corresponding call to `delete` to avoid memory leaks.
3. If you allocate an array using `new[]`, you must deallocate it using `delete[]`.
4. After deleting a pointer, set it to `nullptr` to avoid accidentally dereferencing a dangling pointer.

```
#include <iostream>  
  
int main() {  
    int* arr = new int[5]; // Dynamically allocate an array of 5 integers  
    // Assign values to the array    for  
(int i = 0; i < 5; i++) {        arr[i] =  
i + 1;  
    }  
  
    // Output the values    for (int i  
= 0; i < 5; i++) {        std::cout <<  
arr[i] << " ";  
    }    std::cout <<  
std::endl;  
    delete[] arr; // Deallocate the array
```

```

arr = nullptr; // Set array pointer to nullptr
return
0;
}

```

Pointers to Objects

In C++, **pointers to objects** allow you to dynamically allocate and manipulate objects in memory. By using pointers, you can control the lifetime of objects and access their members dynamically.

```

#include<iostream>
using namespace std;
class Complex{    int
real , imaginary ;
public:
    void getData(){        cout<<"The
real part is "<<real<<endl;
        cout<<"The imaginary part is "<<imaginary<<endl;
    }
    void setData(int a , int
b){        real = a;
imaginary = b;
    }
};
int main(){
    Complex *ptr = new Complex;
    (*ptr).setData(1,53);
    (*ptr).getData();
    delete
ptr;
return 0;
}

```

Arrays of objects:-

```

/*
Syntax for initialization list in constructure:

*/
#include<iostream>

```

```

using namespace std;
class Complex{    int
real , imaginary ;
public:
    void getData(){        cout<<"The
real part is "<<real<<endl;
        cout<<"The imaginary part is "<<imaginary<<endl;
}
    void setData(int a , int
b){        real = a;
imaginary = b;
    }
};
int main(){
    Complex *ptr = new Complex[3];
    for(int i = 0;i<3 ;
i++){
        (*(ptr + i)).setData(i , i +1);
        (*(ptr + i)).getData();
    }
    delete
[] ptr;
return 0;
}

```

this pointer

In C++, the **this pointer** is a special pointer that refers to the object for which a member function is called. It is automatically passed to all non-static member functions of a class and can be used to access the calling object's members and methods. The **this** pointer is particularly useful in various scenarios, such as when you need to differentiate between member variables and parameters or when you want to return the current object from a member function.

Key Points about the **this** Pointer:

- It is available only in **non-static member functions**.
- It **always points to the current object**.

- It is a **constant pointer**, meaning you cannot change what it points to.
- It helps in **chaining method calls** by returning the current object.
- For an object `obj`, inside the member function, `this` points to `&obj`.

1. Basic Example

The `this` pointer can be used to access the calling object's members and differentiate between local variables and class members that may have the same name.

```
#include <iostream>
using namespace std;
class MyClass
{ private:
    int data;

public:
    // Constructor
    MyClass(int data) {
        // Using 'this' to refer to the class member 'data'      this-
        data = data;
    }

    // Member function to display data      void display() {      cout
    << "Data = " << this->data << endl; // Optional use of 'this'
    }
};
int main() {
    MyClass obj(100); // Create an object
    obj.display();    // Call the display function

    return 0;
}
```

2. Returning the Current Object Using `this`

The `this` pointer can also be used to return the current object, allowing **method chaining** (calling multiple methods on the same object).

```

#include <iostream> using
namespace std;
class MyClass
{ private:
    int data;
public:
    MyClass(int data = 0) {
this->data = data;
    }

    // Setter method using 'this' to return the current
object    MyClass& setData(int data) {        this->data =
data;        return *this;    // Return the current object
    }    void display() {        cout
<< "Data = " << data << endl;
    }
};
int main() {
    MyClass obj;

    // Method chaining: calling multiple methods on the same object
obj.setData(30).display();
    return
0;
}

```

Arrow operator ->

The **arrow operator** (->) in C++ is used primarily with pointers to access members (variables or functions) of a class or structure. It simplifies the syntax when dealing with pointers compared to using the dereference operator (*) along with the dot operator (.).

```
pointer->member
```

Here, `pointer` is a pointer to an object (of a class or struct), and `member` can be a variable or function defined in that class or struct.

Key Points

- The arrow operator combines two operations: dereferencing the pointer and accessing a member.
- It is often used in scenarios involving dynamic memory allocation or when working with data structures like linked lists, trees, etc.

```
#include <iostream>
using namespace std;
class Point
{ public:
    int x, y;

    Point(int x, int y) : x(x), y(y) {} // Constructor
void display() {          cout << "Point(" << x << ", " <<
y << ")" << endl;
    }
};
int main() {
    Point* p = new Point(10, 20); // Dynamically allocate a Point object
    // Accessing members using the arrow operator      cout << "X
coordinate: " << p->x << endl; // Accessing variable x      cout <<
"Y coordinate: " << p->y << endl; // Accessing variable y
    // Calling member function using the arrow operator      p-
>display(); // Output: Point(10, 20)
    delete p; // Free allocated
memory      return 0;
}
```

The arrow operator is a convenient way to access members of an object through a pointer. It simplifies code readability and reduces the chances of errors when dereferencing pointers and accessing members, making it a fundamental operator when working with classes and structures in C++.

Composition

Composition is a fundamental concept in object-oriented programming (OOP) that describes a specific type of relationship between classes. It refers to a "has-a" relationship, where one class (the container) contains objects of another class (the components or parts). Here's a detailed explanation of composition, including its characteristics, advantages, use cases, and examples.

1. Definition of Composition

- **Composition** is a design principle that models a relationship where one class is made up of one or more objects of other classes. The composed objects are integral parts of the container class and cannot exist independently of it.

2. Characteristics of Composition

- **Strong Ownership:** In composition, the lifecycle of the contained objects is tied to the lifecycle of the container object. When the container is destroyed, the contained objects are also destroyed.
- **Encapsulation:** The container class encapsulates the behavior and data of the composed objects, allowing for controlled interaction and data hiding.
- **No Shared Instances:** In composition, the parts (contained objects) are typically created and managed by the container class, leading to unique instances for each container.

3. Syntax and Implementation in C++

To implement composition in C++, you typically include objects of one class within another class. Here's a simple example:

Example: Car and Engine

In this example, we create a `Car` class that contains an `Engine` class, demonstrating a composition relationship.

```

#include <iostream> using
namespace std;

// Engine class
class Engine { private:
    int horsepower;
public:
    Engine(int hp) : horsepower(hp) {}
    void start()
    {
        cout << "Engine with " << horsepower << " horsepower started." <<
endl;    }    void stop() {        cout << "Engine stopped." << endl;
    }
};

// Car class that contains an
Engine class Car { private:
    Engine engine; // Composition: Car "has-a" Engine
public:
    Car(int hp) : engine(hp) {} // Initialize Engine in Car's constructor
    void startCar() {        engine.start(); // Start the
engine when starting the car        cout << "Car is ready to
drive." << endl;
    }    void stopCar() {        engine.stop(); // Stop the
engine when stopping the car        cout << "Car has
stopped." << endl;
    }
};

int main() {
    Car myCar(150); // Create a Car object with an Engine of 150
horsepower    myCar.startCar(); // Start the car    myCar.stopCar(); //
Stop the car

    return 0;
}

```

4. Advantages of Composition

- **Reusability:** Classes can be reused in different contexts by composing them with other classes.
- **Flexibility:** Composition allows for dynamic changes to the parts of a class at runtime, unlike inheritance, which is static.
- **Encapsulation:** Each class can encapsulate its functionality and state, reducing dependencies and improving maintainability.
- **Easier Testing:** Since composed classes are separate entities, they can be tested independently.

5. Use Cases for Composition

- **Real-World Modeling:** When modeling complex systems where components have clear ownership and lifecycle ties (e.g., a `Library` class that contains `Books`).
- **Replacing Inheritance:** When you want to provide behavior without creating rigid class hierarchies (favoring composition over inheritance).
- **Dynamic Behavior:** When you want to change behavior at runtime by swapping out components (e.g., a game character that can have different weapons).

Aggregation

Aggregation is a fundamental concept in object-oriented programming (OOP) that describes a specific type of relationship between classes. It represents a "has-a" relationship where one class (the whole) contains references to objects of another class (the parts), but the parts can exist independently of the whole.

1. Definition of Aggregation

- **Aggregation** is a relationship where a class (the container) holds references to one or more objects of another class (the components). Unlike composition, the components can exist independently of the container. The lifecycle of the components is not strictly tied to the lifecycle of the container.

2. Characteristics of Aggregation

- **Weaker Relationship:** In aggregation, the contained objects (parts) can exist independently of the container object (whole). If the container is destroyed, the contained objects may still exist.

- **Shared Ownership:** Components can be shared between different containers, meaning they can be part of multiple aggregations.
- **Encapsulation:** Similar to composition, aggregation allows the encapsulation of behaviors and attributes of the contained classes.

```
#include <iostream>
#include <string>
using namespace std;

class Instructor {
private:
```



```

        string name;
int phone_no;
public:
    Instructor(string n = "", int ph = 0)
{
    name = n;        phone_no = ph;
    void print() {    cout << "Instructor name
is " << name << endl
        << "Phone no is " << phone_no << endl;
    }
}; class
TextBook {
private:
    string title;
int book_id;
public:
    TextBook(string t = "", int id = 0)
{
    title = t;        book_id =
id;
    }    void print() {        cout << "Book
name is " << title << endl
        << "Book id is " << book_id << endl;
    }
};
class Course { private:
    string course_name;
    Instructor* inst; // Pointer to an Instructor
    TextBook* book;   // Pointer to a TextBook
public:
    // Constructor that accepts pointers to Instructor and TextBook
Course(string course = "", Instructor* instructor = nullptr, TextBook*
textbook = nullptr) {        course_name = course;        inst =
instructor; // Assign the instructor reference book = textbook;    //
Assign the textbook reference
    }

```

```

    void setCourse(string course = "", Instructor* instructor =
nullptr, TextBook* textbook = nullptr) {        course_name = course;
inst = instructor; // Assign the instructor reference        book =
textbook; // Assign the textbook reference    }    void print() {
cout << "Course name is " << course_name << endl;        if (inst) {
inst->print(); // Use the pointer to call print
    }        if (book) {            book->print();
// Use the pointer to call print
    }
    }
};
int main() {
    Instructor instructor("John Doe", 1234567890);
    TextBook textbook("C++ Programming", 101);

    Course course("Programming 101", &instructor, &textbook); // Pass
existing objects    course.print(); // Print course details
    return
0;
}

```

Polymorphism

Polymorphism in C++ is a core concept of Object-Oriented Programming (OOP) that allows objects of different classes to be treated as objects of a common base class. It enables a single function, operator, or object to behave differently based on the context or the data types it is operating on. There are two main types of polymorphism in C++:

1. Compile-Time Polymorphism (Static Polymorphism):

This type of polymorphism is resolved during compilation. It is usually implemented using:

- **Function Overloading:** Multiple functions with the same name but different parameter lists.

- **Operator Overloading:** Defining new behaviors for C++ operators when applied to objects of user-defined types.

```
#include <iostream>

using namespace std;
class Printer
{ public:
    void print(int i) {          cout <<
"Printing int: " << i << endl;
    }    void print(double d) {          cout <<
"Printing double: " << d << endl;
    }
};

int main() {
Printer p;
    p.print(5);          // Calls print(int)
    p.print(5.5);        // Calls print(double)
return 0;
}
```

2. Run-Time Polymorphism (Dynamic Polymorphism):

This type of polymorphism occurs during runtime and is typically achieved through **inheritance** and **virtual functions**. With runtime polymorphism, a base class pointer or reference can point to derived class objects, and the appropriate method is called based on the actual derived class type.

Key Concepts:

- **Virtual Functions:** Functions in the base class are declared with the `virtual` keyword, which allows derived classes to override them.
- **Overriding:** When a derived class defines its own version of a virtual function

```
#include<iostream>

#include<string> using namespace
std;
class Sahoo{    protected:
    string title;
```

```

        float rating;

public:
    Sahoo(string s, float
r){        title = s;
rating = r;
    }
        virtual void
display(){}
}; class Video : public
Sahoo{    int videoLength;
public:
    Video(string s, float r , int vl): Sahoo(s,r) {
videoLength = vl;
    }    void display(){        cout<<"Title is
video "<<title<<endl            <<"Rating of
video is "<<rating<<endl
        <<"Video length is "<<videoLength<<endl;
    }
}; class Text : public
Sahoo{    int words_Count;
public:
    Text(string s, float r , int wc): Sahoo(s,r) {
words_Count = wc;
    }
    void display(){
        cout<<"Title of text is "<<title<<endl
<<"Rating of text is "<<rating<<endl
        <<"Word count is "<<words_Count<<endl;
    }
};

int main(){
    Sahoo *base[2];
    Video vd("Sahoo",4.9,4);

```

```

    base[1] = &vd;    base[1]-
>display();

    Text txt("Sahoo Text",5,19);
base[2] = &txt;    base[2]-
>display();    return 0;
}

```

Key Takeaways:

- **Compile-Time Polymorphism:** Achieved using function or operator overloading. The decision on which function to call is made at compile time.
- **Run-Time Polymorphism:** Achieved through virtual functions and inheritance. The decision on which function to call is made at runtime.

Polymorphism allows flexibility and scalability in designing software, especially when dealing with class hierarchies. It helps in writing more general and reusable code by decoupling the method call from the actual implementation.

Abstraction

In C++, **abstraction** is one of the key concepts of Object-Oriented Programming (OOP). It refers to the practice of hiding the internal implementation details of a class and exposing only the necessary functionalities to the user. This helps in reducing complexity and allows the user to interact with objects at a higher level.

Key Aspects of Abstraction in C++:

1. **Interface and Implementation Separation:**
 - The user of a class doesn't need to know how the methods work internally, just what they do. This is achieved through class definitions that expose the methods (public interface) but keep the implementation details hidden (private members).
2. **Access Modifiers:**
 - **Private:** Members that are hidden from the outside and can only be accessed from within the class.
 - **Public:** Members that are accessible from outside the class.
 - **Protected:** Members that are accessible in derived classes but not outside the class.
3. **Abstract Classes:**

- Abstract classes in C++ are classes that cannot be instantiated on their own and are meant to provide a base for other classes. An abstract class contains at least one **pure virtual function**, which is a function declared with = 0. This ensures derived classes must provide implementations for these functions, creating a layer of abstraction.

```
#include <iostream>

using namespace std;

// Abstract class
class Shape {
public:
    // Pure virtual function providing interface framework.
    virtual void draw() = 0;
    virtual double area() = 0;
};

// Derived class - Circle
class Circle : public Shape {
private:
    double radius; public:
    Circle(double r) : radius(r) {}
    void draw() override {
        cout << "Drawing a Circle" << endl;
    }
    double area() override {
        return 3.14159 * radius * radius;
    }
};

// Derived class - Rectangle class
Rectangle : public Shape {
private:
    double width, height;
public:
    Rectangle(double w, double h) : width(w), height(h) {}
    void draw()
override {
    cout << "Drawing a Rectangle" << endl;
}
```

```

        double area() override {
return width * height;
        }
};
int main() {
    Shape* shape1 = new Circle(5);
    Shape* shape2 = new Rectangle(4, 6);
    shape1->draw();    cout << "Area of Circle: " <<
shape1->area() << endl;
    shape2->draw();    cout << "Area of Rectangle: " <<
shape2->area() << endl;
    delete
shape1;
delete shape2;

    return 0;
}

```

Explanation:

- The `Shape` class is an abstract class that defines a generic interface for shapes with the `draw` and `area` methods.
- The classes `Circle` and `Rectangle` implement the `Shape` interface by providing concrete implementations for `draw` and `area`.
- The abstraction allows us to interact with `Shape` objects in a uniform way, without worrying about the details of each specific shape.

This encapsulates the idea of abstraction, where the complexity of each shape's specific behavior is hidden behind the common interface.

Interfaces

In the context of object-oriented programming (OOP), an **interface** is a way to define a contract or a set of functionalities that a class must implement. While C++ does not have a specific keyword for interfaces like some other languages (e.g., Java or C#), it achieves the concept of interfaces using **abstract classes** with **pure virtual functions**.

Key Concepts of Interfaces:

1. Pure Virtual Functions:

- A pure virtual function is a function that has no implementation in the base class and must be implemented by any derived class.
- It is declared by assigning = 0 at the end of the function declaration in the base class.

2. Abstract Classes:

- If a class contains at least one pure virtual function, it is considered an abstract class, and it cannot be instantiated directly. ○ The abstract class acts as an interface, defining what derived classes must implement.

3. Multiple Inheritance:

- In C++, a class can implement multiple interfaces by inheriting from multiple abstract classes, thus allowing multiple inheritance.

4. Implementation of Interfaces:

- Any class that inherits from an abstract class and provides definitions for all of its pure virtual functions is said to implement the interface.

```
#include <iostream>

using namespace std;

// Abstract class
class Shape {
public:
    // Pure virtual function providing interface framework.
    virtual void draw() = 0;
    virtual double area() = 0;
};

// Derived class - Circle
class Circle : public Shape {
private:
    double radius;
public:
    Circle(double r) : radius(r) {}
    void draw() override {
        cout << "Drawing a Circle" << endl;
    }
    double area() override {
        return 3.14159 * radius * radius;
    }
}
```

```

};

// Derived class - Rectangle class
Rectangle : public Shape {
private:
    double width, height;
public:
    Rectangle(double w, double h) : width(w), height(h) {}
    void draw() override {        cout
<< "Drawing a Rectangle" << endl;
    }        double area()
override {        return
width * height;
    }
};

int main() {
    Shape* shape1 = new Circle(5);
    Shape* shape2 = new Rectangle(4, 6);
    shape1->draw();        cout << "Area of Circle: " <<
shape1->area() << endl;
    shape2->draw();        cout << "Area of Rectangle: " <<
shape2->area() << endl;
    delete
shape1;
delete shape2;

    return 0;
}

```

Operator overloading

Operator overloading in C++ is a feature that allows you to define or modify the behavior of operators (like +, -, *, <<, etc.) for user-defined types (such as classes and structs). By overloading an operator, you can enable these operators to work with objects in a way that feels intuitive and natural, similar to how they work with built-in types.

Key Concepts of Operator Overloading:

1. **Syntax:** Operator overloading is done by defining a special function with the keyword `operator` followed by the operator you want to overload. This function can either be a member function of a class or a non-member function.
2. **Types of Operators You Can Overload:**
 - Most operators can be overloaded in C++, including arithmetic (+, -, *, /), relational (==, <, >), logical (&&, ||), and others like [], (), ->, etc.
 - Some operators, like . (dot), ?: (ternary conditional), and :: (scope resolution), cannot be overloaded.
3. **Unary and Binary Operators:**
 - **Unary operators** (like ++, --, !, ~, etc.) operate on one operand and can be overloaded.
 - **Binary operators** (like +, -, *, /, ==, etc.) operate on two operands and can also be overloaded.
4. **Friend Functions:** Some operators may need to be overloaded as non-member (friend) functions if the left-hand operand is not of the class type.

Exercise:-

Write a implementation of a class named Binary having two data (a and y) of type integer with private access. Data member of this class should 1 and 0 (default value) for a particular object.

Q1: Write default and parameterized constructors. Q2: Write a standard implementation of overloaded arithmetic operator(*=).

Solution:-

```
#include <iostream> using
namespace std;
class
Binary{ int
x , y;
public:
    Binary(): x(1) , y(0) {}
    Binary(int xValue , int yValue) : x(xValue) , y(yValue){}
    Binary& operator*=(int
multiplier){      x *=multiplier;
y *=multiplier;

    return *this;
}      void display(){
cout<<"x: "<<x<<" y: "<<y<<endl;
```

```

    }
};
int main() {
    Binary b1(2,4);
    b1 *= 3;
    b1.display();
}

```

Without this pointer:-

```

#include <iostream> using
namespace std;
class Binary
{
    int x,
y;
public:

    Binary() : x(1), y(0) {}

    Binary(int xValue, int yValue) : x(xValue), y(yValue) {}

    Binary operator*=(int multiplier) {

        Binary result(x * multiplier, y * multiplier);
return result;
    }

    void display() const {
cout << "x: "
<< x << ", y: " << y << endl;
    } };
int main() {
    Binary b1(2, 4);
    b1 = b1 *= 3;
    b1.display();
return 0;
}

```

Templates

In C++, **templates** are a powerful feature that allows writing generic and reusable code. They enable functions and classes to operate with different data types without rewriting code for each type. There are two primary kinds of templates in C++:

1. Function Templates

A function template allows the same function to be written for different data types. Here's a basic example:

```
#include <iostream>
using namespace std;
// Function template
template <typename T> T
add(T a, T b) {
    return a + b;
}

int main() {
    cout << "Sum of integers: " << add(3, 5) << endl;    // Integer addition
    cout << "Sum of doubles: " << add(2.5, 4.1) << endl; // Floating point addition
    return 0;
}
```

In this example, the function `add` works for both `int` and `double` because the template allows it to be called with different types.

2. Class Templates

A class template provides the ability to create classes that can handle different data types. Here's a simple example of a class template for a stack:

```

#include <iostream> using
namespace std;
template <typename
T> class Pair {
private:
    T first, second;
public:

    Pair(T a, T b) : first(a), second(b) {}

    void display() {          cout << "First: " << first << ",
Second: " << second << endl;
    }
}; int
main() {

    Pair<int> intPair(10, 20);
intPair.display();

    Pair<double> doublePair(3.14, 2.71);
doublePair.display();
    return
0;
}

```

Example:-

PUACP

```

#include<iostream>
using namespace std;
template <class
T> class vector{
public: T *arr; int size;
vector(int m) : size(m){
arr = new T[size];
}

T dotProduct(vector &v){
T d = 0;          for (int i = 0; i <
size; i++)
{
d += this->arr[i] * v.arr[i];
}
return d;

}
~vector(){
delete arr;
}

}; int
main(){
vector<float>
v1(3);    v1.arr[0] =
4.3;    v1.arr[1] =
3.4;    v1.arr[2] =
7.6;

vector<float> v2(3);
v2.arr[0] = 1.0;    v2.arr[1]
= 0;    v2.arr[2] = 1.9;
float a = v1.dotProduct(v2);
cout<<a<<endl;    return 0;
}

```

Above example for double:-

```
int main(){
    vector<double>
v1(3);    v1.arr[0] =
4.3;    v1.arr[1] =
3.4;    v1.arr[2] =
7.6;

    vector<double> v2(3);
v2.arr[0] = 1.0;    v2.arr[1] =
0;    v2.arr[2] = 1.9;
double a = v1.dotProduct(v2);
cout<<a<<endl;    return 0;
}
```

Template for multiple parameters

In C++, function templates with multiple parameters can be written by specifying multiple template type parameters. Below is an example of how you can create a function template that handles multiple types of parameters:

```

#include<iostream>
#include<string> using
namespace std;
template <class T1 , class T2> class myClass{
public:    T1 data1;    T2 data2;    myClass(T1
d1 , T2 d2) : data1(d1) , data2(d2){}    void
display(){        cout<<this->data1<<" "<<this-
>data2<<endl;
    }
}; int main(){    myClass<int, string>
c1(1,"Id, Hello");    c1.display();
myClass<int, float> c2(13, 2.4);
c2.display();    return 0;
}

```

Template with default parameters

In C++, you can define templates with default type parameters, allowing you to specify default types that can be used if no specific type is provided during instantiation. This feature is useful for simplifying code when common types are often used.

```

#include<iostream>
using namespace std;

template <class T1 = int , class T2 = float
> class Salaar{    public:    T1 a ;
    T2 b;
    Salaar(T1 x , T2
y){        a = x;
b = y;    }
    void display(){        cout<<"The
value of a is "<<a<<endl

```

```

        <<"The value of b is "<<b<<endl;
    }
};
int main(){
    Salaar<> s(1 ,3.2);
    s.display();
return 0;
}

```

Key Points:

- Default template parameters simplify code and reduce the need for specifying types when using commonly used ones.
- This flexibility allows you to use the same template class with different types while still having sensible defaults when needed.

Member Function template & overloading template

In C++, you can create member function templates as well as overload template functions. This allows for flexibility and reusability in your code. Let's explore both concepts with examples.

1. Member Function Templates

A member function template allows you to define a function within a class that can operate on any data type. Here's how to create a class with a member function template.

```

#include <iostream> using
namespace std;

template <typename T>
class Calculator { public:
    // Member function template for adding two
values    T add(T a, T b) {        return a + b;
    }

    // Member function template for multiplying two
values    T multiply(T a, T b) {    return a * b;

```

```

    }
};
int main() {
    Calculator<int> intCalc;        cout << "Int Add: " <<
intCalc.add(5, 3) << endl;        cout << "Int Multiply: " <<
intCalc.multiply(5, 3) << endl;
    Calculator<double> doubleCalc; cout << "Double Add: "
<< doubleCalc.add(5.5, 3.2) << endl;
    cout << "Double Multiply: " << doubleCalc.multiply(5.5, 3.2) << endl;
    return
0;
}

```

2. Overloading Template Functions

You can also overload template functions. This means you can have multiple functions with the same name but different parameters or different types.

```

#include <iostream>
using namespace std;

// Function template to find the maximum of two
values template <typename T> T findMax(T a, T b) {
return (a > b) ? a : b;
}

// Overloaded function template for three
values template <typename T> T findMax(T a, T
b, T c) {    return findMax(findMax(a, b), c);
} int main() {    cout << "Max of 3 and 7: " << findMax(3, 7) << endl;
cout << "Max of 3.5 and 2.1: " << findMax(3.5, 2.1) << endl;    cout <<
"Max of 3, 5, and 1: " << findMax(3, 5, 1) << endl;    cout << "Max of 2.5,
3.1, and 0.5: " << findMax(2.5, 3.1, 0.5) << endl;
    return
0;
}

```

Summary

- **Member Function Templates:** Allow you to define member functions that can work with different data types.
- **Overloading Template Functions:** Enables multiple versions of a function with the same name but different parameters, increasing flexibility in your code.

Both features are part of the power of C++ templates, allowing you to write more generic and reusable code.

Standard Template Library (STL)

The **STL (Standard Template Library)** is a powerful set of C++ template classes to provide common data structures and algorithms. It is part of the C++ Standard Library and provides efficient implementations for commonly used structures like arrays, linked lists, stacks, queues, and algorithms like sorting, searching, etc. The STL is divided into three main components:

1. Containers

Containers store collections of objects. They manage data and provide ways to store, access, and manipulate data efficiently.

Types of Containers:

- **Sequence Containers:** Maintain the order of insertion.
 - `vector`: Dynamic array, similar to arrays but can resize dynamically.
 - `deque`: Double-ended queue, allows insertion and deletion at both ends.
 - `list`: Doubly linked list, allows fast insertion and deletion from anywhere.
- **Associative Containers:** Use keys for fast data access.
 - `set`: A collection of unique elements, usually ordered.
 - `map`: Stores key-value pairs, with unique keys.
 - `mset`: A collection that allows duplicate elements.
 - `multimap`: A map that allows multiple values for the same key.
- **Unordered Containers:** Similar to associative containers but with no particular order and generally faster access using hashing.
 - `unordered_set`
 - `unordered_map`
 - `unordered_multiset`
 - `unordered_multimap`

```
#include <iostream>
#include <vector>
```

```
int main() {    std::vector<int> vec = {1,
2, 3, 4, 5};
    // Iterate and print
    for (int val : vec) {
        std::cout << val << " ";
    }
    return 0;
}
```

2. Iterators

Iterators are used to point to elements in containers and are similar to pointers. They provide an interface for navigating through container elements. They enable algorithms to work on a wide range of container types in a uniform manner.

□ Types of Iterators:

- **Input/Output Iterator:** Reads/writes data once.
- **Forward Iterator:** Moves forward through a container.
- **Bidirectional Iterator:** Moves forward and backward.
- **Random Access Iterator:** Moves arbitrarily through the container.

```
#include <iostream>

#include <vector>

int main() {    std::vector<int> vec =
{10, 20, 30, 40};
    // Using iterator to access elements
    std::vector<int>::iterator it;
    for (it = vec.begin(); it != vec.end(); ++it) {
        std::cout << *it << " "; // Dereferencing iterator to get
value    }
    return 0;
}
```

3. Algorithms

Algorithms are functions that operate on containers via iterators. These include searching, sorting, modifying data, and more.

- **Common algorithms:**

- **sort**: Sorts elements.
- **find**: Finds an element in a range.
- **reverse**: Reverses the order of elements.
- **accumulate**: Performs cumulative operations on elements.

```
#include <iostream>

#include <vector>
#include <algorithm> // for sort

int main() {    std::vector<int> vec = {50,
10, 40, 30, 20};

    // Sort the vector
    std::sort(vec.begin(), vec.end());

    // Print sorted vector
    for (int val : vec) {
        std::cout << val << " ";
    }
    return 0;
}
```

Advantages of STL:

- **Reusability**: Provides well-tested, efficient implementations of common data structures and algorithms.
- **Generic Programming**: Works with any data type (using templates), which adds flexibility.
- **Efficiency**: Algorithms and data structures are optimized for performance.

Summary:

STL is an essential part of modern C++ programming. It simplifies the use of data structures and algorithms, enabling developers to write more efficient and cleaner code without reinventing common components.

Exceptional handling in cpp

Exception handling is a mechanism for handling runtime errors or exceptional conditions that might occur during the execution of a program. C++ provides the following keywords for handling exceptions:

- **try:** A block where you write code that might throw an exception.
- **throw:** Used to throw an exception when an error occurs.
- **catch:** A block that handles the exception thrown by `throw`.

```
try {  
    // Code that might throw an exception  
} catch (exception_type  
e) {  
    // Code to handle the exception  
}
```

```
#include <iostream>  
#include <stdexcept>  
using namespace std; // Use the standard  
namespace  
{  
    int divide(int a, int b) {  
        if (b == 0) {  
            throw  
runtime_error("Division by zero!"); // Throwing an exception  
        }  
        return a / b; // Perform  
division  
}  
int main() {  
    try {  
        int result = divide(10, 0); //  
This will throw an exception  
        cout << "Result: " << result  
<< endl; // Output the result  
    }  
    catch (const runtime_error& e) {  
        cerr << "Error: " <<  
e.what() << endl; // Handle the exception  
    }  
    return 0;  
}
```

Explanation:

- **Throwing an exception:** In the `divide` function, if the divisor is zero, a `std::runtime_error` is thrown using the `throw` keyword.
- **Catching the exception:** In the `main` function, the `divide` function is called inside a `try` block. If an exception is thrown, the control passes to the corresponding `catch` block, where the error message is displayed using `e.what()`.

Types of Exceptions:

1. **Standard Exceptions:** C++ provides a set of standard exceptions derived from the `std::exception` class. Some commonly used standard exceptions include:
 - o `std::runtime_error`
 - o `std::logic_error`
 - o `std::out_of_range`
 - o `std::invalid_argument`
2. **Custom Exceptions:** You can create your own exception classes by inheriting from `std::exception` or other standard exception classes.

Key Concepts:

- **throw:** Used to signal that an error has occurred.
- **try and catch:** Used to handle the exception when it occurs.
- **std::exception::what():** Provides an error message when catching exceptions.

Best Practices for Exception Handling:

1. **Use exceptions for exceptional cases:** Avoid using exceptions for normal control flow.
2. **Catch exceptions by reference:** It is good practice to catch exceptions by reference to avoid object slicing and ensure proper polymorphism.
3. **Throw exceptions by value:** This avoids unnecessary copying and ensures optimal performance.
4. **Resource management:** Ensure that resources (e.g., file handles, memory) are released properly even when exceptions occur. Use RAII (Resource Acquisition Is Initialization) to manage resources in C++.

